Scuola universitaria professionale della Svizzera italiana
Dipartimento tecnologie innovative

SUPSI

Messterpiece

Gruppo 5

Studente/i

Algisi Gabriele, Cereni Fabio, Molteni Federico, Pernisi Patrick Lapo

Corso di laurea	Modulo / Codice Progetto	Anno
Game Development	M-I5170Z.1	2025/26

Data

19-01-2026

STUDENTSUPSI

Introduzione

- Progetto di semestre Game Development
- Sviluppato con Unity
- Tema “painting”
- Link alla repo GitLab

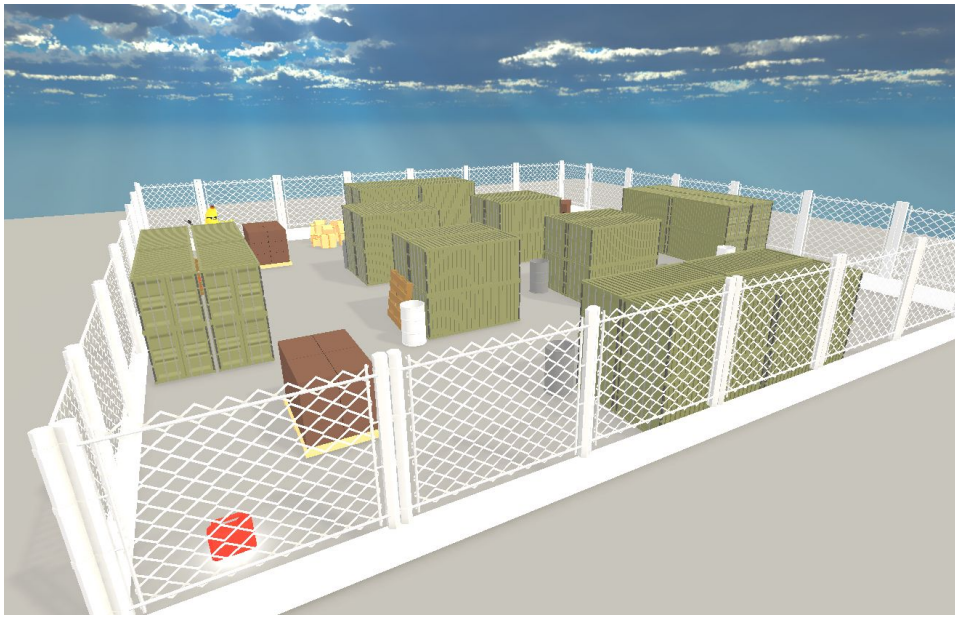
STUDENTSUPSI

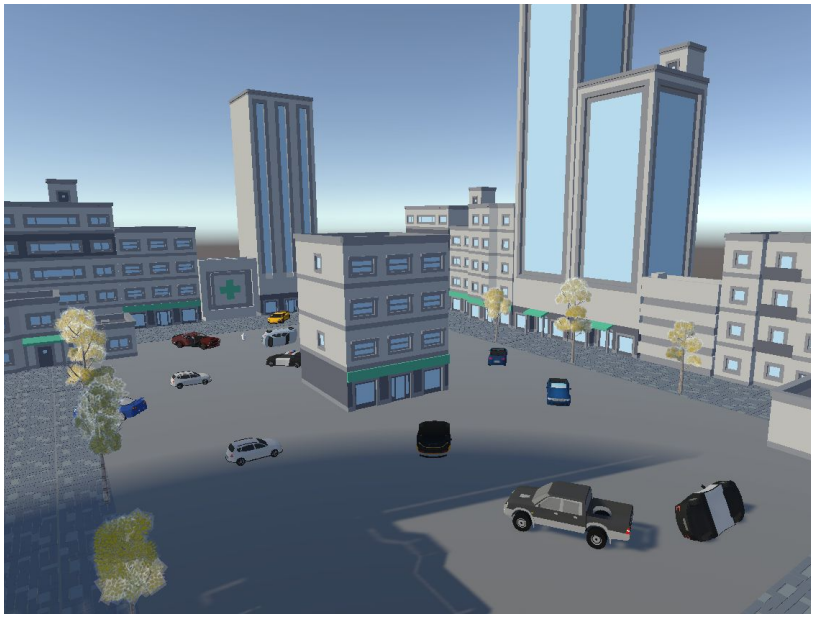
Gameplay

- FPS a tema paintball
- Team Deatmatch: player vs NPC
- 3 mappe

STUDENTSUPSI

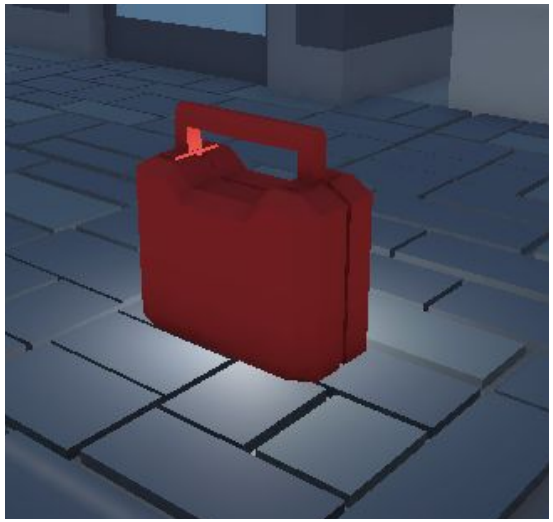
Mappe





Altre feature

- 1a e 3a persona
- Tutorial
- HUD
- Leaderboard/punteggi
- Punti di respawn casuali
- Suoni
- Medikit per curare il giocatore
- Menu delle opzioni
  - Suono
  - Difficoltà (danno e HP di NPC)



STUDENTSUPSI

Dettagli implementativi: camera terza persona

```
private float CheckCameraCollision(Vector3 origin, Vector3 direction)
{
    RaycastHit hit;
    float checkDistance = targetDistance + collisionRadius;

    // Esegui SphereCast (direzione invertita perché andiamo verso la camera, non verso il player)
    if (Physics.SphereCast(/* args */)
    {
        isColliding = true;

        // Calcola distanza sicura dal muro
        float safeDistance = hit.distance - collisionRadius - minDistanceFromWall;
        safeDistance = Mathf.Max(safeDistance, minDistanceFromWall);

        // Smooth zoom
        targetCollisionDistance = Mathf.Lerp(
            targetCollisionDistance,
            safeDistance,
            Time.deltaTime * collisionZoomSpeed
        );

        return targetCollisionDistance;
    }
    else
    {
        // Nessuna collisione
    }
}
```

STUDENTSUPSI

Dettagli implementativi: patrol del nemico

```
void FindClosestTarget()
{
    // Trova tutti i potenziali bersagli nel raggio di rilevamento
    Collider[] potentialTargets = Physics.OverlapSphere(
        transform.position,
        detectionRadius,      // 15 metri
        targetLayers          // Layer "Player" e "Enemy"
    );

    foreach (var col in potentialTargets)
    {
        // Ignora sé stesso
        if (col.transform.root == transform.root) continue;

        // Calcola la direzione verso il bersaglio
        Vector3 directionToTarget = (col.transform.position - transform.position).normalized;

        // Calcola l'angolo tra "dove sto guardando" e "dove è il bersaglio"
        float angle = Vector3.Angle(transform.forward, directionToTarget);

        // Il bersaglio è nel cono visivo?
        if (angle < fieldOfViewAngle / 2f)  // 90° / 2 = 45° per lato
        {
            // Controlla che non ci siano muri in mezzo
            if (!Physics.Raycast(..., obstacleLayer))
            {
                // Bersaglio valido!
            }
        }
    }
}
```

STUDENTSUPSI

Dettagli implementativi: combattimento del nemico

```
private void HandleCombat()
{
    // 1. SEMPRE guarda verso il bersaglio
    FaceTarget(currentTarget.position);

    float dist = Vector3.Distance(transform.position, currentTarget.position);

    // 2. CONTROLLO DISTANZA (Anti-girotondo)
    if (dist <= attackStoppingDistance)  // 6 metri
    {
        // Troppo vicino → FERMATI e spara
        agent.isStopped = true;
        animator.SetBool("isRunning", false);
    }
    else
    {
        // Lontano → Corri verso il bersaglio
        agent.isStopped = false;
        agent.speed = runSpeed;
        agent.SetDestination(currentTarget.position);
        animator.SetBool("isRunning", true);
    }

    // 3. LOGICA DI SPARO
    Vector3 aimDir = (currentTarget.position - transform.position).normalized;
    float aimAngle = Vector3.Angle(transform.forward, aimDir);

    // Spara solo se:
    // - Stiamo guardando verso il bersaglio (entro 10°)
    // - Il cooldown del fuoco è finito
    // - Non ci sono muri in mezzo
    if (aimAngle < shootingAngleThreshold && Time.time >= nextFireTime)
    {
        if (!Physics.Raycast(firePoint, aimDir, dist, obstacleLayer))
        {
            // Mira al petto (+0.8 su Y)
            Vector3 shootDirection = (currentTarget.position + Vector3.up * 0.8f -
firePoint).normalized;
            enemyGun.Shoot(shootDirection);
            nextFireTime = Time.time + fireRate;  // Cooldown 1 secondo
        }
    }
}
```

Dettagli implementativi: Match Flow Manager

```
private IEnumerator WarmupSequence()
{
    IsWarmupActive = true;

    // Blocca il cursore
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;

    // Inizia in BIANCO E NERO
    colorAdjustments.saturation.value = -100f;

    float elapsedTime = 0f;

    while (elapsedTime < countdownDuration)  // 5 secondi
    {
        elapsedTime += Time.deltaTime;
        float progress = elapsedTime / countdownDuration;  // 0.0 → 1.0

        // Transizione da -100 (B/N) a 0 (colori)
        colorAdjustments.saturation.value = Mathf.Lerp(-100f, 0f, progress);

        // Aggiorna il countdown UI
        /* ... */
    }

    // Mostra "GO!" per mezzo secondo
    yield return new WaitForSeconds(0.5f);

    IsWarmupActive = false;
    OnWarmupComplete?.Invoke();  // Notifica altri sistemi
}
```

Demo

STUDENTSUPSI

Conclusioni

- primo tentativo di implementare un gioco da zero
  - diverso tempo per prendere la mano con Unity
- gioco completo che fornisce una base su cui costruire
- collaborazione e gioco di squadra hanno favorito la creatività

STUDENTSUPSI